

PyHelmholtz

version 0.1.10

A Python package for solving the 2D Helmholtz equation and benchmarking absorbing boundary methods.

Features

- Fast frequency-domain wave propagation modeling.
- Support for standard sparse direct solvers via SciPy.
- Optional high-performance nested dissection algorithms via SuiteSparse.
- Optional parallel direct solver acceleration via MUMPS.

Installation

PyHelmholtz provides three installation tiers depending on your performance needs and platform.

1. Standard Installation (Recommended)

If you only need standard solvers using SciPy's `spsolve()`, you can install PyHelmholtz instantly with no external C-library or compiler requirements:

```
pip install pyhelmholtz
```

2. With Nested Dissection Support (Optional)

To enable advanced nested dissection algorithms, the package requires `scikit-sparse`. Because this depends on underlying SuiteSparse C-libraries, it is highly recommended to install the dependencies via Conda first to avoid compilation issues (especially on Windows):

```
# Install the C-libraries and pre-compiled wrapper via Conda
conda install -c conda-forge scikit-sparse

# Install PyHelmholtz with the optional suitesparse feature flag
pip install pyhelmholtz[suitesparse]
```

3. With MUMPS Acceleration (Optional, Linux only)

For large-scale problems requiring parallel direct solvers, PyHelmholtz interfaces with MUMPS via PyMUMPS. Because this relies on compiled Fortran, MPI, and BLAS/LAPACK binaries, you must install the system MUMPS libraries first before installing the Python package:

```
# Install system MUMPS and MPI development libraries (Ubuntu/Debian)
sudo apt-get update
sudo apt-get install libmumps-dev

# Install PyHelmholtz with the optional mumps feature flag
pip install pyhelmholtz[mumps]
```

Quick Start

Here is a quick example of how to use PyHelmholtz to solve the Helmholtz equation using the default parameters:

- Homogeneous domain of size $[-1, 1] \times [-1, 1]$ m² with wave speed of 3×10^8 m/s and grid spacing of 1 cm.
- Point source of frequency 2×10^9 Hz located at the center of the domain.
- The second-order Engquist-Majda absorbing boundary condition (EM2) is used to prevent boundary reflections.
- Second-order finite-difference (FD) schemes are used.

```
import pyhelmholtz as ph

ho = ph.Helmholtz() # create a Helmholtz object with default parameters
ho.solve() # SciPy's spsolve() is used by default as the sparse linear solver
ho.viz() # visualize the real part of the wave field u
```

Here is another example in which several parameters were manually set. These include the grid spacing $h = 10$ m, the horizontal and vertical domain limits = $[0, 1000]$ and $[0, 500]$ respectively, the wave speed = 1000 m/s, the point source frequency = 10 Hz, and location = $[500, 250]$. In addition, the perfectly matched layer (PML) is used as the absorbing boundary method, and the fourth-order FD schemes are used in the calculation.

```
import pyhelmholtz as ph

domain = ph.Domain(h=10, limits=[0, 1000, 0, 500], v=1000)
source = ph.PointSource(freq=10, xs=500, ys=250)
ho = ph.Helmholtz(domain=domain, abm=ph.PML(), source=source, fd=ph.FD(4))

ho.solve()
ho.viz()
```